UNIT-5

Link HTML, CSS & JS file in Express JS

To use HTML, CSS, JS files, we can create separate folders to store HTML & CSS files and separate folder to store JS files with other package.json files.

Methods:

express.static(root, [options…])

Express provides a built-in middleware **express.static(folder_name)** to serve static files, such as HTML,images, CSS, JavaScript, etc. Here, folder_name is a root directory to serve static assets.

app.use()

This function is used to mount the specified middleware function(s) at the path which is being specified. It is mostly used to set up middleware for your application.

Syntax: app.use(path, callback)

path: It is the path for which the middleware function is being called. It can be a string representing a path or path pattern or a regular expression pattern to match the paths.
callback: It is a middleware function or a series/array of middleware functions.

You simply need to pass the name of the directory where you keep your static assets, to the **express.static** middleware to start serving the files directly.

Serving static files in Express

- ✓ To serve static files such as images, CSS files, and JavaScript files, use the express.static built-in middleware function in Express.

The function signature is:

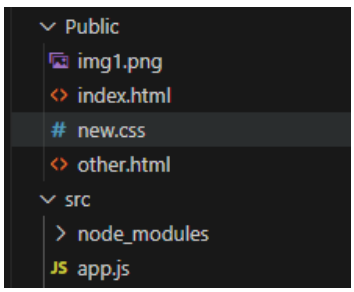
express.static(root, [options])

The root argument specifies the root directory from which to serve static assets. For more information on the options argument, see express.static.

For example, if you keep your images, CSS, and JavaScript files in a directory named public, you can do this

For example: app.use(express.static('../public'));

We will keep a few images and HTML file in **public** sub-directory as follows –



```
var express = require('express');
var app = express();
app.use(express.static( '../public'));
 app.listen(8081, () =>{
   console.log("server start")})
```

Save the above code in a file named **app.js** and run it with the following command.

```
$ node app.js
```

Now open **localhost:8081/img1.png** in any browser and see observe your Image on web page.

Note:

***path.join(paths)[Concept of node]** : This method joins the specified path segments into one path.

**The path that you provide to the express.static function is relative to the directory from where you launch your node process. If you run the express app from another directory, it's safer to use the absolute path of the directory that you want to serve.

***We can also provide a path prefix for serving static files. For example, if you want to provide a path prefix like '/static' in app.get("/static", C.B).

app.use('/static', express.static('public')).

****__dirname** is an environment variable that tells you the absolute path of the directory containing the currently executing file.

Folder structure

All Files are in One Folder	Industry Approach	All Logic files are in src folder	Non Logic files are in public folder
Express **|-----1.js** **|-----index.html** **|-----1.css** **|-----1.png**	**Express** **|-------src** **|---------|---1.js** **|-------public** **|---------|--- index.html** **|---------|---1.css** **|---------|---1.png**	**Express** **|------src** **|--------|--1.js** **|---index.html** **|---1.css** **|---1.png**	**Express** **|---1.js** **|-----public** **|------|---index.html** **|------|---1.css** **|------|---1.png**
Code			
app.use (expr.static ('./')) Using "_ _ dirname" **app.use (expr.static (__dirname))**	**app.use (express.static('../public'))** **Here we can also join the path** **const sp = path.join (__dirname ,"../public");** **app.use (expr.static(sp));**	**app.use (express.static("../"))** **Here we can also join the path** **const sp = path.join (__dirname ,"../");** **app.use(expr.static(sp));**	**app.use (expr.static("public"))**

Example-1:**Load HTML file named index.html (create a public folder inside express folder and create index.html and express.css inside it. Also, create src folder and create 1.js file inside it.)**

Express
|-------src
|---------|---1.js
|-------public
|---------|--- index.html
|---------|---1.css
|---------|---1.png

/public/index.html	**/public/express.css**	**/src/1.js**

<html>
<head>
<link rel="stylesheet" href="express.css">>
</head>
<body>
 File name must be index.html
</body></html>
	.lj_p {
	font-size:100;
	color:blueviolet;
	}
		const expr = require ("express")
		const app = expr();
		
		// From here it will load HTML file from public folder but name of file must be index.html
		
		app.use(expr.static("../public"))
		app.listen (5200)

Output in browser:

File name must be index.html

Load HTML file on home page

```
const expr = require("express");
const app = expr();
var sp=app.use(expr.static("../public"))
app.get('/', (req, res) => {
   res.sendFile(sp + "/index.html");
});
app.listen(5200, () => {
   console.log("Server running on http://localhost:5200");
});
```

//Another way to load html file using sendFile() and path.join()

```
const expr = require ("express")
const path = require ("path");
const app = expr();
const staticpath = path.join (__dirname,"../public");
app.get('/',(req,res)=>{
res.sendFile(staticpath +"/index.html”);
})
app.listen (5200)
```

Output: File name must be index.html (without CSS)

Note: Here css file **will not be loaded** automatically. So, add **app.use(expr.static(staticpath));**

After assigning staticpath variable is preferred where dependency stack is present in files. Means to Serve static files (CSS, JS, images, etc.)

Example 2 (File Name is not index.js)

/public/2.html	**/public/express.css**	**/src/2.js**

<head>
<link rel="stylesheet" href="express.css">>
</head>
<body>
 File name can be different.
</body>
	.lj_p {
	font-size:100;
	color:blueviolet;
	}
		const expr = require ("express")
		const path = require ("path");
		const app = expr();
		app.use (expr.static("../public",{index:"other.html"}))
		// If name is different then index.html
		app.listen(5200)

Output in browser: File name can be different.

Example: understand concept of all files in different folders.

Express
|-------src
|---------|---app.js
|-------html
|---------|---1.html
|-------css
|---------|---1.css
|-------image
|---------|--- 1.png

1.html

```
<html>
<head>
<link rel="stylesheet" href="1.css">
   </head>
   <body>
      <p class="p"> Hello </p>
      <img src="1.png">
   </body>
</html>
```

1.css

```
.p{
    font-size: 30px;
    color:tomato;
    text-align: center;
}
img{ width:400px; height:auto;}
```

app.js

```
const express = require("express");
const app = express();
app.use(express.static("../css"))
app.use(express.static("../img"))
app.use(express.static("../html",{index:"1.html"}))
app.listen(5006)
```

Using path module

```
const express = require("express");
const app = express();
const path= require("path");

var css_path = path.join(__dirname,"../css")
var img_path= path.join(__dirname,"../image")
var html_path = path.join(__dirname,"../html")

app.use(express.static(css_path))
app.use(express.static(img_path))
app.use(express.static(html_path,{index:"1.html"}))
app.listen(5001,()=>console.log("Started"))
```

Note that if you don't want to use path module to join the path then you can use below code.

```
app.use(express.static("../public"));
 OR
app.use(express.static(__dirname + "/../public"));
```

Problem without path

- May break on different systems
- Path may not resolve correctly
- Not reliable for real projects

Handling Forms & Form Data

- Handling forms and form data means sending user input from the browser to the server. This is done using HTTP methods, mainly GET and POST. When a user fills a form and submits it, the browser sends a request to the server.
- The GET method is used when the data needs to be retrieved from the server. In this method, the form data is sent through the URL as a query string. The data appears after the question mark in the URL. Because the data is visible in the URL, it is not secure. The amount of data that can be sent using GET is limited. In Express, the data sent using GET is accessed using req.query. GET is commonly used for search operations, filters, and fetching data where security is not important.
- The POST method is used when the data needs to be sent to the server for processing or storage. In this method, the data is sent inside the request body and is not visible in the URL. This makes POST more secure compared to GET. POST can handle large amounts of data. In Express, the data sent using POST is accessed using req.body, but it requires middleware such as express.json or express.urlencoded to parse the data. POST is commonly used for form submissions like login, registration, and uploading data.
- The main difference between GET and POST is how and where the data is sent. GET sends data through the URL, while POST sends data through the request body. GET is faster and simple but less secure, whereas POST is more secure and suitable for sensitive or large data.

GET method

The GET method requests a representation of the specified resource. Requests using GET should only retrieve data and should have no other effect. Get method facilitates you to send only limited amount of data because data is sent in the header. It is not secure because data is visible in URL bar. It facilitates you to send only limited amount of data because data is sent in the header. It is not secure because data is visible in URL bar.

Example: Write express script to get form data using get method and display data in JSON format in next page named “process_get”.

public/index.html

```
<html>
<body>
<form action="/process_get" method="get">
First Name: <input type="text" name="first_name"> <br>
Last Name: <input type="text" name="last_name">
<input type="submit" value="Submit">
</form>
</body>
</html>
```

src/example1.js

```
var express = require('express');
var app = express();
app.use(express.static("../public"))
app.get('/process_get', function (req, res) {
var fn = req.query.first_name;
var ln = req.query.last_name
  console.log(req.query);  //req.query is used to get the submitted data in json object format. To access particular key value we have to write "req.query.first_name"
  res.send(`Welcome ${fn} ${ln}`);
})
app.listen(8000)
```

Example: Write express js script to print message in next line splitting by "." And use get method to submit the data. HTML file contains form of text area for the message and submit button.

public/2.html

```
  <form method="get" action="/login">
      <textarea rows="10" cols="10" name="message">Hi.Hello.how are you
      </textarea>
      <button name="Submit" value="Submit">Submit</button>
   </form>
```

src/2.js

```
var expr = require("express");
var app = expr();
app.use(expr.static("../public",{index: '2.html'}));
app.get("/login",(req,res)=>{
   res.set("content-type","text/html");

   t1 = (req.query.message).split(".");
   for(i in t1){
      res.write(t1[i]+ "</br>");
   }
   res.send();
});
app.listen(5121,()=>{
   console.log("server start");
});
```

Example 3

Write express js script to perform tasks as asked below.

1. **Create one HTML file which contains two number type input fields, one dropdown which contains options like (select, addition, subtraction, multiplication, division) and one submit button.**
2. **The input fields must contain the value greater than 0 else it will give a message "Please enter the valid number". Also, user must select any of the formula from the dropdown else give a message "You have not selected any formula". (Message will be displayed on "/calc" page.)**
3. **If one formula is selected and numbers are entered then respective calculations will be performed on the page "/calc".**
4. **Use get method to request data.**

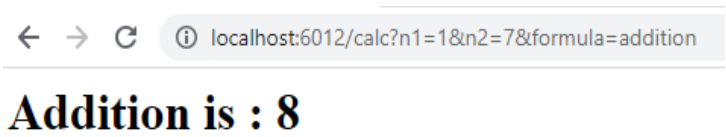
calc.js

```
var express = require("express");
var app = express();
app.use(express.static(__dirname,{index:'calc.html'}));
app.get("/calc",(req,res)=>{
   res.set("content-type","text/html");
   var n1 = parseInt(req.query.n1);
   var n2 = parseInt(req.query.n2);
   if ((n1>0) && (n2>0)) {
      if(req.query.formula == "addition"){
         a = n1+n2;
         res.write("<h1>Addition is : " + a + "</h1>");
      }
      else if(req.query.formula == "subtraction"){
         s = n1-n2;
         res.write("<h1>Subtraction is : " + s + "</h1>");
      }
      else if(req.query.formula == "multi"){
         m = n1*n2;
         res.write("<h1>Multiplication is : " + m + "</h1>");
      }
      else if(req.query.formula == "div"){
         d = n1/n2;
         res.write("<h1>Division is : " + d + "</h1>");
      }
      else{
         res.write("<h1>You have not selected any formula.</h1>");
      }
   }else{
      res.write("<h1>Please enter the valid number/s.</h1>");
   }
   res.send()
})
app.listen(6012);
```

calc.html

```
<html>
<body>
   <form action="/calc" method="get">
   <fieldset>
      <legend>Calculator:</legend>
         <input type="number" name="n1"><br><br>
         <input type="number" name="n2"><br><br>
          <select name="formula" >
            <option value="">Select formula</option>
            <option value="addition">Addition</option>
            <option value="subtraction">Subtraction</option>
            <option value="multi">Multiplication</option>
            <option value="div">Division</option>
         </select><br><br>
         <input type="submit" value="Submit">
   </fieldset>
   </form>
</body>
</html>
```

Output:



Addition is : 8

POST method

The POST method requests that the server accept the data enclosed in the request as a new object/entity of the resource identified by the URI. Post method facilitates you to send large amount of data because data is sent in the body. Post method is secure because data is not visible in URL bar.

express.urlencoded()

The **urlencoded()** function is a built-in middleware function in Express.
It parses incoming requests with URL-encoded payloads and is based on a body parser.

- express.urlencoded() expects request data to be sent encoded in the URL, usually in strings or arrays **{extended:Boolean Value}**

- **extended :** The extended option allows to choose between parsing the URL-encoded data with the **querystring** library (**when false**) or the **qs** library (**when true**).

Specify the extended option as either true or false. Here's what that means:

- extended: false → Uses the **querystring** library (doesn't support rich objects).
- extended: true → Uses the **qs** library (allows rich objects and arrays).

Defaults to true, but using the default has been deprecated.

Initial Line

```
var expr = require("express");
app.use(expr.urlencoded({extended:true})); // for qs library
```

or

```
app.use(express.urlencoded({ extended: false }));// for querystring library.
```

*****Note:**

- New version of express contains built-in body-parser module(As it is third party module).
- For Express vesion of 4.0 above We don't need to install it and import it to our program.
- We can access it's functionality by just adding below line to our program.

Installation * (Only required in old version)**

npm install body-parser

In code Initilization syntax

```
var bodyParser = require('body-parser')
app.use(bodyParser.urlencoded({ extended: false }));
```

- Body-parser parses is an HTTP request body that usually helps when you need to know more than just the URL being hit.

- It provides four express middleware for parsing JSON, Text, URL-encoded, and raw data sets over an HTTP request body.
- Using body-parser allows you to access req.body from within routes and use that data.

Example 1: Write express script to get form data using post method and display data in JSON format in next page named "process".

public/p.html

```
<html>
<body>
<form action="/process" method="post">
First Name: <input type="text" name="first_name">  <br>
Last Name: <input type="text" name="last_name">
<input type="submit" value="Submit">
</form>
</body>
</html>
```

src/p.js

```
var express = require('express');
var app = express();
app.use(express.urlencoded({ extended: false }));
app.use(express.static("../public",{index: 'p.html'}));
app.post('/process', function (req, res) {
  res.send(req.body);
})
 app.listen(7002)
```

Example 2

Build an Express.js app with:

1. **An HTML form with username, password, and submit button. Use POST method.**
2. **On visiting /, show the form.**
3. **On submitting to /login:**
   - **If username is "admin", display: "Welcome admin"**
   - **Else, show: "Please login with admin name" and a link back to the form.**

e2.html

```
<form method="post" action="/login">
   <div class="lj-div">
      <label>User name:</label>
      <input type = "text" name="uname" ></input>
   </div>
   <div class="lj-div">
      <label>Password:</label>
      <input type = "text" name="password" ></input>
```

```
    </div>
    <button name="Submit" value="Submit">Submit</button>
</form>
```

e2.js

```
var express = require("express");
var app = express();
app.use(express.urlencoded());
app.use(express.static(__dirname,{index: 'e2.html'}));
app.post("/login",(req,res)=>{
   if(req.body.uname == 'admin'){
      res.write("<h1> Welcome" + req.body.uname + "</h1>")
   }
   else{
       res.write(`<h1 style="color:red">Please login with admin name</h1> <a href='/'>Try
again</a>`);
   }
   res.send();
})
app.listen(5222);
```

Example: Write express js script to perform the tasks as asked below.

1) Create one HTML file named ljform.html and add one form which contains username, password and submit button. Data should be submitted by HTTP post method.

2) Submit button is of black color with white text. (External CSS)

3) On home page form should be displayed and while submitting the form, on next page named "/login" if username is admin then it will display "Welcome admin" else display "Please login with Admin name".

src/3.js

```
var expr = require("express");
var app = expr();

app.use(expr.urlencoded());
app.use(expr.static('../public',{index: ' ljform.html'}));
app.post("/login",(req,res)=>{
   if( req.body.uname == 'admin' ){
      res.write(`<h1 style="color:green">Hey ${req.body.uname},Welcome!</h1><br>`)
   }
   else{
      res.write(`<h1 style="color:red">Please login with Admin name</h1>`);
   }
   res.send();
```

```
})
app.listen(5222);
```

public/ ljform.html

```
<html>
   <head><title>Post Method</title>
      <link rel="stylesheet" href="1.css">
   </head>
   <body>
      <form method="post" action="/login">
      <div class="lj-div">
         <label>User name:</label>
         <input type = "text" name="uname" ></input>
      </div>
      <div class="lj-div">
         <label>Password:</label>
         <input type = "text" name="password" ></input>
      </div>
      <button name="Submit" value="Submit">Submit</button>
   </form>
   </body>
</html>
```

CSS file is also in public folder

```
button{
   padding: 10px;
   background-color: black;
   color: #fff;}
```

Cookie

Cookies are small piece of information i.e. sent from a website and stored in user's web browser when user browses that website. Every time the user loads that website back, the browser sends that stored data back to website or server, to recognize user.

How cookies work

When a user visits a cookie-enabled website for the first time, the browser will prompt the user that the web page uses cookies and request the user to accept cookies to be saved on their computer. Typically, when a makes a user request, the server responds by sending back a cookie (among many other things).

This cookie is going to be stored in the user's browser. When a user visits the website or sends another request, that request will be sent back together with the cookies. The cookie will have certain information about the user that the server can use to make decisions on any other subsequent requests.

For example, **Facebook** from a browser.
When you want to access your Facebook account, you have to log in with the correct credentials. When you first make a login request and the server verifies your credentials, the server will send your Facebook account content. It will also send cookies to your browser. The cookies are then stored on your computer and submitted to the server with every request you make to that website. A cookie will be saved with an identifier that is unique to that user.

When you revisit Facebook, the request you make, the saved cookie, and the server will keep track of your login session and remember who you are and thus keep you logged in.

As explained, cookies are stored in the browser, and no sensitive information can be stored in them. They are typically used to save a user's preferences.

- **cookie-parser** - cookie-parser looks at the headers in between the client and the server transactions, reads these headers, parses out the cookies being sent, and saves them in a browser. In other words, cookie-parser will help us create and manage cookies depending on the request a user makes to the server.

Run the following command to install these NPM packages:

```
npm install cookie-parser
```

We will create a simple example to demonstrate how cookies work.

Step 1 - Import the installed packages

To set up a server and save cookies, import the cookie parser and express modules to your project. This will make the necessary functions and objects accessible.

```
const express = require('express')
const cookieParser = require('cookie-parser')
```

Step - 2 Get your application to use the packages

You need to use the above modules as middleware inside your application, as shown below.

```
//setup express app
const app = express()
// let's you use the cookieParser in your application
app.use(cookieParser());
```

This will make your application use the cookie parser and Express modules.

Step -3 Add/Read/Delete Operation

- **Add/Set Cookie: res.cookie()** method is used for setting the cookie name to value. The value parameter can be a string or an object converted to JSON.

  Syntax: res.cookies("name","value","expiry-date")

- **Display cookie:** To view cookies from your server req.cookies function is used. addthe following console to that route.

  console.log('cookies:' ,req.cookies)

- **Delete cookie** : The **res.clearCookie()** function is used to clear the cookie specified by name. This function is called for clearing the cookies which has already been set. For example, if a user cookie is set, then it can be cleared using this function.

  Syntax:res.clearCookie("cookie-name")

How Long Does a Cookie Store?

The time of expiry of a cookie can be set when the cookie is created. If time is not set by the user then by default, the cookie is destroyed when the current browser window is closed.

Adding Cookies with Expiration Time:-

You can add cookies that expire. Just pass an object with property 'expire' and set to the time when you want it to expire from your browser. Following is an example of this method.

// res.cookie(name, 'value', {expires:new Date(Date.now()+10000)}); Expires after 10000 ms from the time it is set.

The other way to use{ maxAge: oneDay } - this sets the cookie expiry time. The browser will delete the cookie after the set duration elapses. The cookie will not be attached to any of the requests in the future. In this case, we've set the maxAge to a single day as computed by the following arithmetic.

// res.cookie(name, 'value', {maxAge: 360000}); This cookie also expires after 360000 ms from the time it is set.

Cookie Methods & Options in Express.js

Feature / Method	Type	Description
res.cookie(name, value, options)	Method	Sets a cookie in the browser with optional settings like expiry, httpOnly.
res.clearCookie(name, options)	Method	Deletes a cookie by setting its expiration to the past.
req.cookies	Property	An object containing cookies sent by the browser (requires cookie-parser).

Cookie Expiry Options

Option	Type	Description
maxAge	Number (ms)	Cookie lifetime in **milliseconds** (from now).
expires	Date object	Sets an **exact date & time** for the cookie to expire.

Example: Add/Read/Delete cookie
app.js

```
var express = require('express');
var app = express();
var cp = require('cookie-parser');
app.use(cp());

app.get('/cookie', function(req, res){
res.cookie('name', 'Express JS');
res.cookie('fname', 'express'); //Sets fname = express
res.cookie('lname', 'js'); //Sets lname = js
res.cookie('ID','2',{ expires:new Date(Date.now()+10000)}); //expires after 10 secs
res.cookie('email', 'express@gmail.com',{maxAge:2000}); //expires after 2 secs

res.clearCookie('fname');

//show the saved cookies
res.send(req.cookies);  //Use req.cookies method to check the saved cookies
});
app.listen(3000);
```

When the above route is executed from a browser, the client sends a get request to the server. But in this case,
the server will respond with a cookie and save it in the browser.

Open http://localhost:3000/cookie your browser and access the route.

To confirm that the cookie was saved, go to your browser's Inspect element > select the application tab > cookies > select your domain URL.

To check if your cookie is set or not, just go to your browser, fire up the console, and enter −

console.log(document.cookie);

Example: Routing of cookie for understanding

```
var expr=require("express")
var app=expr()
var cp=require('cookie-parser')
app.use(cp())
app.get("/",(req,res)=>{
   res.cookie("fname","Khushbu") //Sets fname = Khushbu
   res.cookie("lname","Patel")
   res.cookie("contact","001100",{expires:new Date(Date.now()+10000)})
      //expires after 10 secs
   res.cookie('email', 'express@gmail.com',{maxAge:2000}); //expires after 2 secs
   res.cookie('city', 'ahmedabad');
res.send("Cookie set");
})
app.get("/cookie",(req,res,next)=> {
// To print on Console

  console.log(req.cookies)
  console.log(req.cookies.fname)
  res.write(JSON.stringify(req.cookies));
  res.write(req.cookies.fname);
  res.send()
})
app.get("/clear",(req,res)=>{
// To print coockie on another page → localhost:5001/clear
// const cookies = req.cookies
//for (let prop in cookies) {
//     console.log(prop)
//   res.clearCookie(prop); //Or res.cookie(prop, '', {expires: new Date(0)});
// }
   res.clearCookie("fname")
   res.send(req.cookies)   // Use req.cookies method to check the saved cookies . If want to
write coockie using res.write use JSON.stringify
})
app.listen(5001)
```

Example: Write an express js script to set cookies of submitted values of form. Perform following tasks.

- Create a HTML file which contains a form with fields first name, last name, password and a submit button.
- Once form submitted, store all these entered values to the respective cookies on '/next' page.
- Then redirect user to "/admin" page and clear the cookie set for the last name. Display remaining set cookie values on this page. (Using get method)

In form.html file:

```
    <form method="get" action="/next">
    First Name : <input type = "text" name="uname" ></input>
    Last Name : <input type = "text" name="lname" ></input>
    Password : <input type = "password" name="password" ></input>
    <button name="Submit" value="Submit">Submit</button>
  </form>
  </body></html>
```

In cookie.js file:

```
var expr=require("express")
var app=expr()
var cp=require('cookie-parser')
app.use(cp())
app.use(expr.static('../public',{index: 'form.html'}));
app.get("/next",(req,res,next)=>{
   // const response= {
   //     u:req.query.uname,
   //     p:req.query.password }
   // res.cookie("uname",response.u)
   res.cookie("uname",req.query.uname)
    res.cookie("password",req.query.password)
                    // Only for ref*
                    //res.cookie("password", req.query.password ,{ secure: true, httpOnly: true })
                    //always use true value when you want cookies to be created on an HTTPS s
                    //secure origin.
   res.cookie("lname",req.query.lname);
   res.redirect("/admin");  //  alternative of next()*
})
app.get("/admin",(req,res)=>{
   res.clearCookie('lname');
   res.write(" Welcome : " + req.cookies.uname);
   res.write(" Lname : " + req.cookies.lname);
   res.send()
});
app.listen(6959);
```

Example

Write an express js script to set cookies of submitted values of form. Perform following tasks.

- **Create a HTML file which contains a form with fields first name, last name, password and a submit button.**
- **Once form submitted, store all these entered values to the respective cookies on '/next' page.**
- **Then redirect user to "/admin" page and clear the cookie set for the last name. Display remaining set cookie values on this page. (Using post method)**

c1.html

```
<form method="post" action="/next">
   First Name : <input type = "text" name="fname" ></input>
   Last Name : <input type = "text" name="lname" ></input>
   Password : <input type = "password" name="password" ></input>
   <button name="Submit" value="Submit">Submit</button>
</form>
```

c1.js

```
var express = require("express");
var app = express();
var cp = require("cookie-parser");
app.use(cp());
app.use(express.static(__dirname,{index: 'c1.html'}));
app.use(express.urlencoded())
app.post("/next",(req,res,next)=>{
   res.cookie("fname", req.body.fname);
   res.cookie("password", req.body.password)
   res.cookie("lname", req.body.lname);
   res.redirect("/admin");
})
app.get("/admin",(req,res)=>{
   res.clearCookie('lname');
   res.write(" Welcome : " + req.cookies.fname);
   res.write(" Lname : " + req.cookies.lname);
   res.write(" Password : " + req.cookies.password);
   res.send()
});
app.listen(3000);
```

```
Welcome : abc Lname : undefined Password : 123
```

We have cleared lname cookie. So, req.cookies.lname is undefined.

Example

You have been assigned to develop a user feedback form for a website using Express.js and cookies.

Implement the following requirements:

- Create a form with the following fields:
  - Name (input field)
  - Email (input field)
  - Message (textarea field)
  - Rating (radio buttons: Bad, Average, Good, Very Good, Excellent)
- When the user submits the form, store their feedback information (name, email, message, and rating) in a cookie named "feedback" that expires in 10 seconds.
- Display a confirmation message to the user after successfully submitting the form & Create a link to display the feedback details stored in the "feedback" cookie.
- When the user clicks to the link, retrieve the feedback information from the cookie and display it on the page also include a link on the feedback details page to Logout.When the user clicks the link, user redirected to home page.
- After, 10 seconds it will give message “No Feedback available” message to user

feedback.js

```
const express = require('express');
const cp = require('cookie-parser');
const app = express();
app.use(cp());
app.use(express.urlencoded());
app.use(express.static(__dirname,{index:'feedback.html'}))

app.post('/submit-feedback', (req, res) => {
  const { name, email, message, rating } = req.body;
  // Create the feedback object
  const feedback = {name,email,message,rating};
  // Set the feedback cookie with a 10-second expiration
  res.cookie('feedback', feedback, { maxAge: 10000 });
  res.send('Thank you for your feedback! <br> <a href="/feedback-details"> Show Feedback
</a>');
});

app.get('/feedback-details', (req, res) => {
  const feedback = req.cookies.feedback;

  if (feedback) {
    res.send(`
      <h1>Feedback Details</h1>
      <p><strong>Name:</strong> ${feedback.name}</p>
      <p><strong>Email:</strong> ${feedback.email}</p>
      <p><strong>Message:</strong> ${feedback.message}</p>
      <p><strong>Rating:</strong> ${feedback.rating}</p>
```

```
    <a href="/" > logout </a>`);
  } else {
    res.send('No feedback available.');
  }
});

app.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

feedback.html

```
<body>
  <h1>User Feedback Form</h1>
  <form action="/submit-feedback" method="POST">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required><br><br>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required><br><br>

    <label for="message">Message:</label>
    <textarea id="message" name="message" required></textarea><br><br>

    <label for="rating">Rating:</label>
    <input type="radio" id="rating1" name="rating" value="Bad" required>
    <label for="rating1">Bad</label>
    <input type="radio" id="rating2" name="rating" value="Average" required>
    <label for="rating2">Average</label>
    <input type="radio" id="rating3" name="rating" value="Good" required>
    <label for="rating3">Good</label>
    <input type="radio" id="rating4" name="rating" value="Very Good" required>
    <label for="rating4">Very Good</label>
    <input type="radio" id="rating5" name="rating" value="Excellent" required>
    <label for="rating5">Excellent</label><br><br>
    <input type="submit" value="Submit">
  </form>
</body>
```

Session

The difference between session and cookie

Session	Cookies
A session is stored at server side	A cookie is stored at client side
It can store a data ranging between 5mb – 10mb	It can only store a data of 4kb
It is destroyed when user logout.	It is destroyed when user closes the page or it will remain until the defined time.
express-session middleware is required to create a session.	It doesn't require any middleware to create a cookie. Express provide built in support.
Session id is used as a identifier.	Key-value pair data is used as a identifier.
The performance of session is slower due to server interaction.	The performance of cookie is faster, as data is stored locally.
It is more secure as data are stored at server side.	It is less secure as data are stored at client side.
It is used for storing user-specific data.	It is used to store user preference data.

How sessions works

When the client makes a login request to the server, the server will create a session and store it on the server-side.

When the server responds to the client, it sends a cookie. This cookie will contain the session's unique id stored on the server, which will now be stored on the client.

Why Use Sessions?

Sessions are used to store information such as User ID over the server more securely, where it cannot be altered. This prevents the information from being tampered with.

In addition to this, sessions can transfer the information from one web page to another in the form of value.

We use this session ID and look up the session saved in the database or the session store to maintain a one-to-one match between a session and a cookie. This will make HTTP protocol connections stateful.

HTTP is stateless; in order to associate a request to any other request, you need a way to store user data between HTTP requests. Cookies and URL parameters are both suitable ways to transport data between the client and the server. But they are both readable and on the client side. Sessions solve exactly this problem. You assign the client an ID and it makes all further requests using that ID. Information associated with the client is stored on the server linked to this ID. Install the using the command:

```
npm install express-session
```

Express-session options and how to use them

To set up the session, you need to set a couple of Express-session options, as shown below.

```
const oneDay = 1000 * 60 * 60 * 24; // creating 24 hours from milliseconds
app.use(sessions({
   secret: "thisismysecrctekey",
   saveUninitialized:true,
   cookie: { maxAge: oneDay },
   resave: false
}));
```

- **secret -** a random unique string key used to authenticate a session. It is stored in an environment variable and can't be exposed to the public. The key is usually long and randomly generated in a production environment.
- **resave -** Forces the session to be saved back to the session store, even if the session was never modified during the request.

  resave = true (Default) - It means when the modification is performed on the session it will re write the req.session.cookie object. Saves session on **every request** (even if not modified).

  resave = false -It will not rewrite the req.session.cookie object. the initial req.session.cookie remains as it is. Saves session **only if modified** (recommended).Setting false **reduces unnecessary session writes** and prevents race conditions.

- **saveUninitialized -** Forces a session that is "uninitialized" to be saved to the store. A session is uninitialized when it is new but not modified. Choosing false is useful for implementing login sessions, reducing server storage usage, or complying with laws that require permission before setting a cookie.

  Uninitialised = false - It means that Your session is only Stored into your storage, when any of the Property is modified in req.session. Saves a session **only if modified** (recommended for login-based apps).Reducing **unnecessary session storage**.Prevents **storing empty sessions** for visitors who don't log in.

Uninitialised = true (Default) - It means that Your session will be stored into your storage Everytime for request. It will not depend on the modification of req.session Saves new but unmodified sessions.

- **cookie:** { maxAge: oneDay } - this sets the cookie expiry time. The browser will delete the cookie after the set duration elapses. The cookie will not be attached to any of the requests in the future.

Examples: Write express js script using session to display how many times a user visited a website. If user is visiting a website for the first time then display "Welcome! Thank you for visiting our website!" else display the count of user (How many times) for that particular session.

```
const expr=require("express");
const app=expr();
const sess=require("express-session");
app.use(sess({
        resave:false,
        saveUninitialized:false,
        secret:"pkp123"
    }));
app.get("/",(req,res)=>{
    if(req.session.page_views) {
        req.session.page_views++;
        res.send(`<h1 style="color:blue;"> You have visited page ${req.session.page_views} times <h1>`);
    }
    else{
        req.session.page_views=1,
        res.send(`<h1 style="color:green;"> Welcome! Thank you for visiting our website!<h1>`);
    }
});
app.listen(8003,()=>{
    console.log("server running at 8003");
});
```

Examples: write a script to meet following requirements:

1) Create index.html file page and open it on localhost. (Form -username,password,login button)
2) After clicking submit button, it should jump on "savesession" page. Store username in session.
3) After saving session, redirect to "fetchsession" page and read value. Put a LOGOUT link button here.
4) Jump on delete session page after clicking LOGOUT button.
5) Destroy the session on this page and redirect to index.html page.

session.js

```
var expr = require("express");
var app = expr();
var es = require("express-session");
app.use(es({
   resave:false,
   saveUninitialized:false,
   secret:"mykey1"
}));
app.use(expr.static("../public",{index:"trial.html"}))
app.get("/savesession",(req,res)=>{
  req.session.uname = req.query.uname;
  req.session.password = req.query.password;
   res.redirect("fetchsession")
})
app.get("/fetchsession",(req,res)=>{
   res.write("<h1>Welcome " + req.session.uname +"</h1>")
   res.write("<a href='/deletesession'>Delete Session</a>")
   res.send();
});
app.get("/deletesession",(req,res)=>{
  req.session.destroy();
  res.redirect('/')

//Also try res.clearCookie('connect.sid')
//res.send("Session Cookie cleared")
});
app.listen(6177);
```

session.html

```
<form method="get" action="/savesession">
   <div><input type = "text" name="uname" placeholder="Username" ></input></div>
   <div><input type = "password" name="password" placeholder="Password"></input></div>
   <input class="lj_in" type="submit" name="submit" value="Submit">
</form>
```

Example

Write a script to meet following requirements:

- **Create session.html file page which contains form (username, password, login button). and open it on localhost.**
- **After clicking submit button, it should jump on "save" page. Store username and password in session.**
- **After saving session, redirect to "fetchdata" page and read value. On this page check authentication of user. User name and password must be "admin" and "admin@123" respectively.**
    - **If this condition is true then display welcome admin and display logout link on this page(fetchdata).**

- **By clicking on logout link user should jump to "destroy" page and destroy the session there and display the message "Session destroyed". And give the link of "login" under that message. By clicking that link user will be redirected to the home page.**
  - **Else display "Please enter valid username and password" and login link on this page(fetchdata).**

session.html

```
<form method="get" action="/save">
  <div><input type = "text" name="uname" placeholder="Username" ></input></div>
  <div><input type = "password" name="pass" placeholder="Password"> </input>
  </div>
  <input type="submit" name="submit" value="Login">
</form>
```

Session.js

```
var express = require("express");
var app = express();
var es = require("express-session");
app.use(es({
   resave:false,
   saveUninitialized:false,
   secret:"lju1"
}));
app.use(express.static(__dirname,{index:'session.html'}));

app.get("/save",(req,res)=>{
   req.session.uname = req.query.uname;
   req.session.password = req.query.pass;
   res.redirect("fetchdata")
})
app.get("/fetchdata",(req,res)=>{
   if(req.session.uname == "admin" && req.session.password=="admin@123"){
      res.write("<h1 style='color:green;'>Welcome Admin</h1>")
      res.write("<a href='/destroy'>Logout</a>")
   }else{
      res.write("<h1 style='color:red;'>Please enter valid username or password</h1><a
href='/'>Login</a>")
   }
   res.send();
});
app.get("/destroy",(req,res)=>{
   req.session.destroy()
    res.write("<h1>Session Destroyed</h1><a href='/'>Login</a>")
    res.send();
});
app.listen(6177);
```

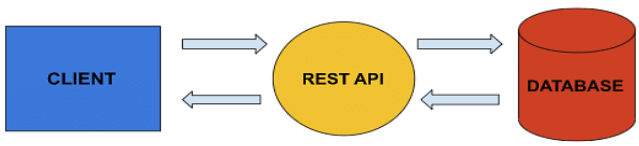
RESTful APIs

REST (**Representational State Transfer)** API is the standard way to send and receive data for web services.

Let's understand the meaning of each word in the REST acronym.

- **REpresentational** means formats (such as XML, JSON, YAML, HTML, etc)
- **State** means data
- **Transfer** means carrying data between consumer and provider using the HTTP protocol

A client sends a req which first goes to the rest API and then to the database to get or put the data after that, it will again go to the rest API and then to the client. Using an API is just like using a website in a browser, but instead of clicking on buttons, we write code to req data from the server. It's incredibly adaptable and can handle multiple types of requests.

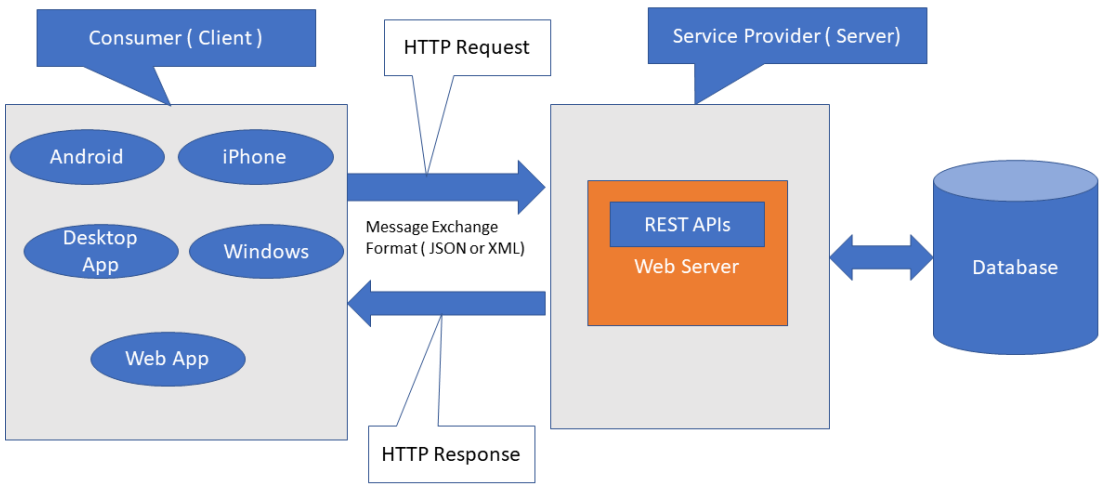


REST (Representational state transfer) is a popular architecture that is used to create web services. Express is a back-end web application framework of node js, and with the help of express, we can create an API very easily.

API (Application Programming Interface) is a code that allows two software programs to communicate with each other.

REST API is a software that allows two apps to communicate with one another over the internet and through numerous devices.

REST Architecture



- **Request and Response:** Request is the input to a web service, and the response is the output from a web service.
- **Message Exchange Format:** It is the format of the request and response. There are two popular message exchange formats: XML and JSON.
- **Service Provider or Server:** The service provider is one that hosts the web service.
- **Service Consumer or Client:** A service consumer is one who is using a web service.

In simple words A RESTful API is an architectural style for an application program interface
(API) that uses HTTP requests to access and use data.

What Is express.Router() For

Express Routers are a way to organize your Express application such that your primary app.js file does not become bloated.
To create an instance of an Express Router, we call the **.Router()** method on the top-level Express import. Then to use that router, we mount it at a certain path using **app.use()** and pass in the router as the second argument. This router willnow be used for all paths that begin with that path segment. To create a router to handle all requests beginning with **/hello**, the code would look like this

```
const helloRouter = express.Router();
app.use('/hello', helloRouter);
```

We will be using JSON as our transport data format as it is easy to work. we are using Router from Express, and we are exporting it using **module.exports**. So, our app will work fine.

Example - 1: Create object mvi in **movies.js** which is serve as **provider** and **index.js** is serve as **consumer**

```
const expr = require("express")
const app = expr()

const mvi = [ {id:101,name:"DON-1",Year:2013,rating:9.5},
 {id:102,name:"DON-2",Year:2016,rating:9.5}, {id:103,name:"DON-3",Year:2023,rating:100}]

app.get("/p",(req,res)=>{
   res.json(mvi)
})
app.get("/p/:id",(req,res)=>{
   const currmovie = mvi.filter(m => m.id == req.params.id)
   if(currmovie.length == 1){
      res.json(currmovie[0])
   } else {
      res.json("not found")
   }
})
app.listen(3015)
```

Method -2 Using expres.Router()

In Index.js (same folder)

```
const expr=require("express")
const app=expr()
const m1=require("./data")
app.use("/p",m1)
app.listen(3015)
```

In data.js

```
const expr=require("express")
const router=expr.Router()
const mvi = [{id:101,name:"DON-1",Year:2013,rating:9.5},
{id:102,name:"DON-2",Year:2016,rating:9.5},
{id:103,name:"DON-3",Year:2103,rating:100}]

router.get("/",(req,res)=>{
   res.json(mvi)
})

//for specific id

router.get("/:id",(req,res)=>{
const currmovie = mvi.filter(m => m.id == req.params.id)

// can use const movie = mvi.find(m => m.id == req.params.id)
//Only if unique item to be found as it return only value not array

if(currmovie.length == 1){
res.json(currmovie[0])
} else {
res.json("not found")
}
})
module.exports=router
```

**Run node index.js in terminal and observe output on :
localhost:3015/p for all movie list and
Localhost:3015/p/103 for specific movie

Example -2

Write an expressJS code in which RESTapi is created for json object named data which contains name,id,branch,city and contact properties.

- On "/api" page it should display all the content.
- Upon passing id on the browser url it should display the content having that id. (i.e. on localhost:7899/api/101)
- Upon passing branch on the browser url it should display the content having that branch. (i.e. on localhost:7899/api/cse)

api.js

```
const expr = require("express");
const router = expr.Router();
const data=[
   { id:101,name:"ABC",branch:"CSE",contact:9876543210,city:"Ahmedabad" },
   { id:102,name:"BCD",branch:"CE",contact:9876543210,city:"Ahmedabad"},
   { id:103,name:"XYZ",branch:"CSE",contact:9876543210,city:"Rajkot" },
   { id:104,name:"PQR",branch:"IT",contact:9876543210,city:"Ahmedabad" },
   { id:105,name:"ABC",branch:"CSE",contact:9876543210,city:"Surat" },
   { id:106,name:"ABC",branch:"IT",contact:9876543210,city:"Rajkot" }]

router.get("/",(req,res)=>{
   res.set("content-type","text/html")
   for(i of data){
      res.write("<h3>ID: "+ JSON.stringify(i.id) + ", Name: " + i.name + ", Branch: " +
i.branch+ ", Contact: " + i.contact + ", City: " + i.city + "</h3>");
   }
   res.send();
})
router.get("/:id",(req,res)=>{
   var current_data = data.filter((i1)=> i1.id == req.params.id)
   if(current_data.length > 0){
      res.send(current_data);}
else{
      res.send("Not Found")
   }
})

router.get("/branch/:branch",(req,res)=>{
   var cd = data.filter((b)=>b.branch.toLowerCase() ==
req.params.branch.toLowerCase())
 if(current_data.length > 0){
      res.send(current_data);
   }else{
      res.send("Not Found")
   }
})
module.exports = router;
```

main.js

```
const expr = require("express");
const app = expr();
const api = require("./api")
app.use("/api",api);
app.listen(7899);
```

To run:
node main.js

Check output in browser:
http://localhost:7899/api

http://localhost:7899/api/101

http://localhost:7899/api/branch/it

Miscellaneous Tasks

Load HTML file named index.html (All files in same folder)

```
express
|-----1.js
|-----index.html
|-----1.css
```

Example

/express/index.html

```
<html>
 <head><link rel="stylesheet" href="1.css"></head>
   <body><p class="lj_p">File name must be index.html </p>
   </body>
</html>
```

/express/1.css

```
.lj_p {
   font-size:100;
   color:blueviolet;
}
```

/express/1.js

```
const express = require ("express")
const app = express();
// name of file must be index.html
app.use(express.static(__dirname))
app.listen (5200)
```

To run in terminal : go to "express" folder by command cd express and then node 1.js

Output in browser:

File name must be index.html

Load HTML file named index.html (create a public folder inside express folder and create index.html and 1.css inside it)

```
express
|---1.js
|-----public
|------|---index.html
|------|---1.css
```

Example

/express/public/index.html

```
<html>
   <head>
     <link rel="stylesheet"
          href="1.css">
   </head>
   <body>
        <p class="lj_p"> File name must be index.html </p>
   </body>
</html>
```

/express/public/1.css

```
.lj_p {
   font-size:100;
   color:blueviolet;
}
```

/express/1.js

```
const express = require ("express")
const app = express();
// name of file must be index.html
app.use(express.static("public"))
app.listen (5200)
```

To run in terminal : go to "express" folder by command cd express and then node 1.js

Output in browser:

File name must be index.html

Load HTML file named index.html (create a public folder inside express folder and create index.html and 1.css inside it. Also, create src folder and create 1.js file inside it.)

```
express
|-------src
|---------|---1.js
|-------public
|---------|---index.html
|---------|---1.css
```

Example

/express/public/index.html	/express/public/1.css	/express/src/1.js

<html>
<head>
<link rel="stylesheet" href="1.css">>
</head>
<body>
 File name must be index.html
</body>
</html>
	.lj_p {
	font-size:100;
	color:blueviolet;
	}
		const express = require ("express")
		const app = express();
		const path = require ("path");
		const **staticpath** = **path.join** (**__dirname**, "../public");
		// name of file must be index.html
		app.use(express.static(staticpath))
		app.listen (5200)

To run in terminal : go to "src" folder by command cd src and then node 1.js

Output in browser:

File name must be index.html

Note that if you don't want to use path module to join the path then you can use below code.

```
const express = require ("express")
const app = express();
app.use(express.static((__dirname, "../public")))
app.listen (5200)
```

Load HTML file named index.html (create index.html and 1.css files inside express folder. Create src folder inside express folder and create 1.js file inside src folder.)

```
express
|------src
|--------|--1.js
|---index.html
|---1.css
```

Example

/express/index.html

```
<html>
   <head>
<link rel="stylesheet" href="1.css"></head>
    <body><p class="lj_p">
File name must be index.html </p>
   </body>
</html>
```

/express/1.css

```
.lj_p {
   font-size:100;
   color:blueviolet;
}
```

/express/src/1.js

```
const express = require ("express")
const app = express();
const path = require ("path");
const staticpath = path.join
(__dirname, "../");
// name of file must be index.html
app.use(express.static(staticpath))
app.listen (5200)
```

To run in terminal : go to "src" folder by command cd src and then node 1.js

Output in browser:

File name must be index.html

load HTML file using res.sendFile()

express/public/1.html

```
<html>
   <head>
   <link rel="stylesheet"
        href="1.css">
   </head>
   <body>
          <p class="lj_p">
Example using sendFile method.</p>
   </body>
</html>
```

express/public/1.css

```
.lj_p {
   font-size:100;
   color:blueviolet;
}
```

express /src/1.js

```
const express = require
("express")
const app = express();
const path = require ("path");
const staticpath = path.join
(__dirname, "../public");

//to load css file which is linked in
html file
app.use(express.static(staticpath))
//Another way to load HTML file
app.get('/',(req,res)=>{
res.sendFile(staticpath +
"/1.html");
 })
app.listen (5200)
```

Load HTML file named other than index.html file.

express/public/2.html	express/public/2.css	express/src/2.js

<html>
<head>
<link rel="stylesheet" href="2.css"> </head>
<body>
 File name is other than index.html.
</body>
</html>
	.lj_p {
	font-size:100;
	color:blueviolet;
	}
		const express = require ("express")
		const path = require ("path");
		const app = express();
		const **staticpath** = path.join (__dirname, "../public");
		app.use (express.static(**staticpath**, {**index:"2.html"**}));
		app.listen(5200)

Middleware Examples with get/post methods

Example 1

Write express js script to perform following tasks.

1. **Create one html file which contains one text field for name, email field and checkbox for subscription. Html file will be loaded on home page. Email and name fields are required fields.**
2. **On login page welcome user and email id data should be printed.**
   a. **If user checked the subscription then "Thank you for the subscription" message will be printed and "logout" link will be displayed under the message. If user clicks logout link then he/she will be redirected to the home page.**
   b. **If user has not opted for the subscription then "You can subscribe to get daily updates" message will be printed and "subscribe" link will be displayed under the message.**
   c. **If user clicks subscribe link then he/she will be redirected to the subscription page. In this page "Thank you for the subscription" message will be printed and "logout" link will be displayed under the message. If user clicks logout link then he/she will be redirected to the home page.**

Use concept of the middleware and you can use any of http methods(get/post).

t1.js

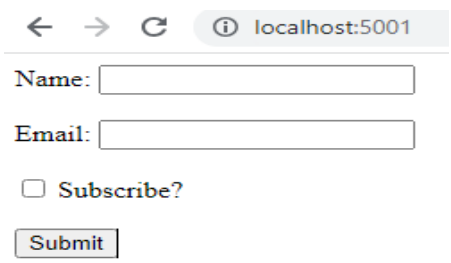
```
var express = require("express");
var app = express();

app.use(express.static(__dirname,{index:'t1.html'}));

app.use(express.urlencoded());

app.post("/login",(req,res,next)=>{
```

```
    res.set("content-type","text/html");
    res.write("<center><h1>Welcome " + req.body.name + "</h1>");
    res.write("<center><h2>Your email id is " + req.body.email + "</h2>");
    next();
});

app.post("/login",(req,res)=>{
    if(req.body.newsletter == "on"){
        res.write("<h3>Thank you for your subscription</h3><a href='/'>Logout</a>");
    }else{
        res.write("<h3>You can subscribe to get daily updates</h3><a
href='/subscribe'>Subscribe</a></center>");
    }
   res.send();
});
app.get("/subscribe",(req,res)=>{
    res.set("content-type","text/html");
    res.write("<h3>Thank you for your subscription</h3></center><a href='/'>Logout</a>");
    res.send();
});
app.listen(5001);
```

t1.html

```
<form action="/login" method="post">
    <label for="name">Name:</label>
    <input type="text" name="name" required><br><br>

    <label for="email">Email:</label>
    <input type="email" name="email" required><br><br>
    <input type="checkbox" name="newsletter" id="newsletter">
     <label for="newsletter">Subscribe?</label><br><br>
     <input type="submit" value="Submit">
</form>
```

Output:



Example 2 : Write an express.js script to load an HTML file having username and password and submit button. On clicking submit button. It should jump on "check" page using "POST" method. If username is "admin" , then jump on next middleware to print "welcome… admin" , if usernameis not "admin" , then stay on same middleware to print "warning msg" in red color.

Index.html

```
<html>
<body>
<form action ="/check" method="post">
Username: <input type="text" name="uname"/>
password: <input type="text" name="pwd"/>
<input type="submit"/>
</form>
</body>
</html>
```

App.js

```
const express=require("express");
const app=express();
app.use(express.urlencoded({extended:false}));
app.use(express.static(__dirname));
app.post("/check",(req,res,next)=>{
if(req.body.uname=="admin")
{
   next();
}
else {
   res.send("<h1 style='color:red'>wrong credentials</h1>") }
});
app.post("/check",(req,res)=>
{
 res.write("<h1>welcome..."+ req.body.uname+ "</h1>")
 res.send();
}
).listen(3001);
```